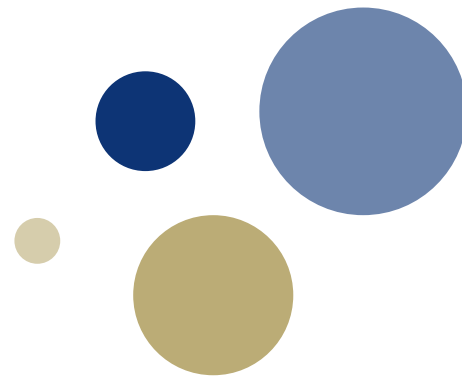


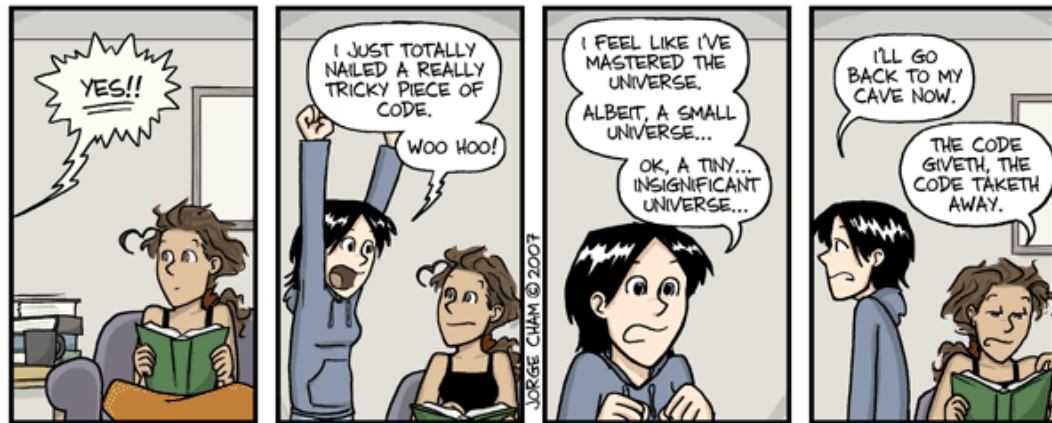


NTNU – Trondheim
Norwegian University of
Science and Technology



Introduction to Pyomo

Kasper E. Thorvaldsen
Adj. Associate Professor
IEL, NTNU
Spring 2025



Optimization in Python -> Pyomo

- «Pyomo is a Python-based open-source software package that supports a diverse set of optimization capabilities for formulating, solving, and analyzing optimization models.”
(cite: <http://www.pyomo.org/about>)
- Very useful optimization tool to solve problems commonly encountered in this field
- Used for solving logical decisions based on typically cost-based considerations
- Used to solve “Bilateral trading game” for pool trading



Pyomo – What it does

- Given a simple optimization problem
 - It allows setting up the problem in Python, and use optimization solvers to solve your problem
 - Variation in how to set up the problem
 - Different complexity
 - Pyomo package must be downloaded, the solvers must also be (some require license!)

$$\begin{aligned} \max_{x \in \mathbf{R}^n} \quad & \sum_{j=1}^n c_j x_j \\ \text{s.t.} \quad & \sum_{j=1}^n a_{ij} x_j \leq b_i, \quad i = 1, \dots, m \\ & x_j \geq 0, \quad j = 1, \dots, n \end{aligned}$$

- Helpful documentation:
<https://pyomo.readthedocs.io/en/stable/>



If stuck with Pyomo:

<https://pyomo.readthedocs.io/en/stable/>



Optimization problem: Resource allocation and production example

Door producing company (Hiller & Lieberman 2010)

A company makes two types of doors,

- Aluminium framed glass doors
 - Selling price: \$3000 a piece
 - Frame takes one hour to make
 - Assembly of frame and glass takes three hours
- Wood framed glass doors
 - Selling price: \$5000 a piece
 - Frame takes two hours to make
 - Assembly of frame and glass also takes two hours

The company rents production capacity at three different factories

- An aluminium frame production facility, available capacity: 4 h/week
- A wood frame production facility, available capacity: 12 h/week
- A frame and glass assembly facility, available capacity: 18 h/week

How can the company decide on how many doors of each type to produce?

Resource allocation and production example

Easy! They formulate this as a linear program.

Door producing company (Hiller & Lieberman 2010)

- Decision variables
 - x_1 : # of aluminium framed glass doors
 - x_2 : # of wood framed glass doors
- Objective function (maximize revenues):
 - Maximize $z = 3000 x_1 + 5000 x_2$
- Constraints:
 - Aluminium frame factory capacity:
 - $x_1 \leq 4$
 - Wood frame factory capacity:
 - $2x_2 \leq 12$
- Frame + glass assembly:
 - $3x_1 + 2x_2 \leq 18$
- (Non-negativity: $x_1 \geq 0$ and $x_2 \geq 0$)

How this is formulated mathematically:

$$\text{Maximize } z = 3000 \cdot x_1 + 5000 \cdot x_2$$

s. t

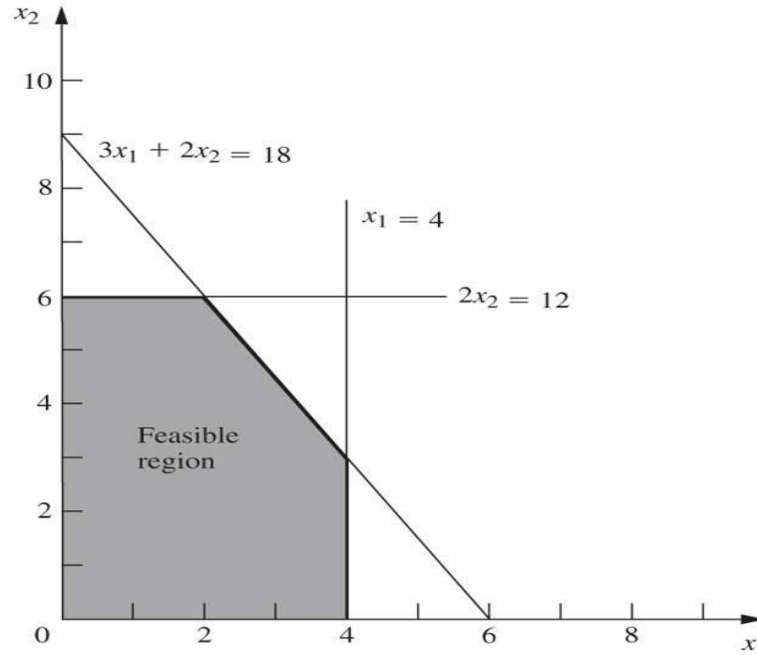
$$x_1 \leq 4$$

$$2x_2 \leq 12$$

$$3 \cdot x_1 + 2 \cdot x_2 \leq 18$$

$$x_1, x_2 \geq 0$$

The feasible region and optimal solution



Pyomo – Creating a model

- Creating a model can be done in several ways:
 - Directly writing the values into the problem
 - Declare parameters based on pre-defined values in the model (all changes are done outside of the model, making it easier to adjust)
 - Use of sets to simplify code
- Let's try creating and solving the problem 2 times!
 - 1 directly with values added
 - 1 with some cleaner methods (will not be covered in this lecture)

- ```
7
8 import numpy as np
9 import sys
10 import time
11 import pandas as pd
12 import pyomo.environ as pyo
13 from pyomo.opt import SolverFactory
```

```
30 #We establish a model variable -> model
31 model = pyo.ConcreteModel() #Create the model variable
32
```

# Pyomo – Creating variables

- Declare variable
  - Specify the variables that we want to optimize
  - Can be dimensionless or be based on sets (this case, dimensionless)
  - Can specify boundaries in the declaration (optional, but can replace constraints later)
    - Can set it to be binary  $\{0,1\}$
    - Can set it to be integer  $\{0,1,2,3,\dots,N\}$
    - Etc

```
#This
model.Alu = pyo.Var(within = pyo.NonNegativeReals)
model.Wood = pyo.Var(within = pyo.NonNegativeReals)
#Note the 'within', this allows us to specify the boundaries of the variable
#This is not necessary, one can specify it as a constraint instead
```

# Pyomo – Declare the Objective

- Objective declared through a function
  - Note: Must have the model as input
  - Return what the objective function is
- For specifying the objective in 'model', one specifies:
  - «rule»: what depicts the objective (write function name)
  - «sense»: What type of optimization is this (min/max)

```
#We need to create a function for this to work, that must include the model variable
def Objective(model):
 #It must return the objective function
 return(3000*model.Alu + 5000*model.Wood)
model.OBJ = pyo.Objective(rule = Objective, sense = pyo.maximize)
#here, we specify the objective into the model variable
#sense specifies what we want to do (max/min). If not declared, it assumes max
#rule specifies what function that operates as the objective function
```

# Pyomo – How to create a constraint

- Create a function specifying the constraint
  - Use of  $\leq$ ,  $\geq$  or  $=$  to specify boundary
- To create constraint in 'model', link the constraint with 'rule'
- Note: Dimensionless or not creates multiple variations (more on this later)
- Can include dimension to create multiple constraints in one declaration (more on this later)

```
#We had several constraints, and must create a function for each one

#Constraint for max time
def Constraint_assemble(model):
 return(3*model.Alu + 2*model.Wood <= 18)
model.C1 = pyo.Constraint(rule = Constraint_assemble)
#Pretty much the same setup as for creating the objective function

def Constraint_AMan(model):
 return(1*model.Alu <= 4)
model.C2 = pyo.Constraint(rule = Constraint_AMan)

def Constraint_WMan(model):
 return(2*model.Wood <= 12)
model.C3 = pyo.Constraint(rule = Constraint_WMan)
```

# Pyomo – Different solvers

- Many solvers exist, and can handle different problems
  - GLPK – Open-source solver, not the best and not very advanced
  - Gurobi – Need license (student license available for free). Very advanced and highly recommended (<http://www.gurobi.com/index>)
  - CPLEX – Need license (student license available for free?). Can handle advanced problems (<https://www.ibm.com/analytics/cplex-optimizer>)
- Packages needed for the solvers:
  - GLPK: `conda install -c conda-forge glpk`
  - Gurobi: `conda install -c gurobi gurobi`
  - Cplex: `conda install -c ibmdecisionoptimization cplex`
  - Note: Packages can be downloaded without license, but trying to use them without is troubling

# Pyomo – How to set a solver and run

- Set a solver
  - Here, GLPK is used
- Enable dual values
  - Good for further analysis, but optional
- Run the problem
  - Solve the problem to get optimal solutions

```
115 ***
116 Specify solver
117 ***
118
119 #We here decide what solver to use, note that you probably need packages for the solvers as well
120
121 #This simple model we use glpk
122 opt = SolverFactory('glpk')
123
124 ***
125 Enable dual values to be stored
126 ***
127
128 #One nice feature is to include extracting dual values. Done with this one line
129 model.dual = pyo.Suffix(direction=pyo.Suffix.IMPORT)
130
131 ***
132 Solve the problem
133 ***
134
135
136 #Solve the problem
137 results = opt.solve(model, load_solutions = True)
138
```

# Pyomo – How to work with results

- Data will be stored in the 'model' variable
- Can get them using .value
- Multiple methods to store the data, which can then be extracted to whatever usage wanted

```
#Solve the problem
results = opt.solve(model, load_solutions = True)

#Display data
model.display()
#Display dual data
model.dual.display()

print(model.Alu.value)
print(model.Wood.value)
```

```
In [20]: runfile('://home.ansatt.ntnu.no/kasperet/Documents/Sci-Ass Courses/2019/TET4185/P
home.ansatt.ntnu.no/kasperet/Documents/Sci-Ass Courses/2019/TET4185/Python-Pyomo/Python i
Model unknown

Variables:
prod : Size=2, Index=Doors
 Key : Lower : Value : Upper : Fixed : Stale : Domain
 Alu : 0 : 2.0 : None : False : False : NonNegativeReals
 Wood : 0 : 6.0 : None : False : False : NonNegativeReals

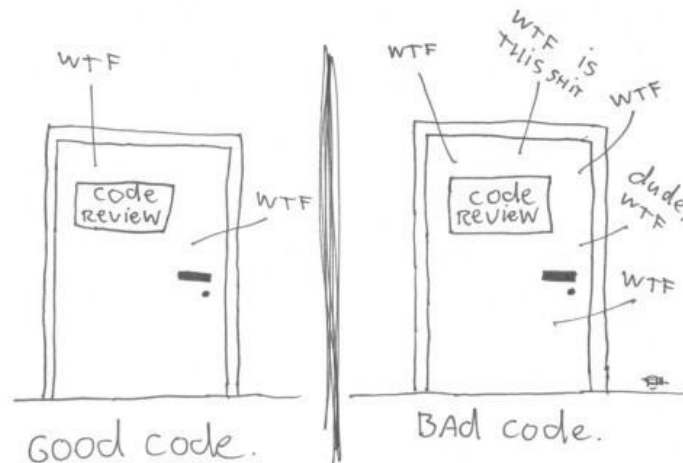
Objectives:
OBJ : Size=1, Index=None, Active=True
 Key : Active : Value
 None : True : 36000.0

Constraints:
MTime_const : Size=1
 Key : Lower : Body : Upper
 None : None : 18.0 : 18.0
FTime_const : Size=2
 Key : Lower : Body : Upper
 Alu : None : 2.0 : 4.0
 Wood : None : 12.0 : 12.0
dual : Direction=Suffix.IMPORT, Datatype=Suffix.FLOAT
 Key : Value
 FTime_const[Alu] : 0.0
 FTime_const[Wood] : 1500.0
 MTime_const : 1000.0
number of Alu doors produced: 2.0
number of Wood doors produced: 6.0
Optimal solution: 36000.0
```

# Pyomo – Problems with this application

- We want to change some data, what do we do?
- We want to include a new type of door, rebuild model?
- -> Create a more general model!
  - A model that is only dependent on the input-data provided!

The ONLY valid measurement  
OF code QUALITY: WTFs/minute





**Pyomo and optimization will be used in future lectures**



**Questions, comments? 😊**



# Pyomo – Working with setup

- Packages needed:
  - Pyomo environment
  - Solver package
  - Note the naming of the packages (pyo, opt)
- Specify input data
  - Can be done in different ways (see attached file for more)
- Declare model
  - Variable 'model' is now declared, but needs a skeleton to work

```
7
8 import numpy as np
9 import sys
10 import time
11 import pandas as pd
12 import pyomo.environ as pyo
13 from pyomo.opt import SolverFactory
```

```
15 #What type of doors do we have
16 Doors = ["Alu", "Wood"]
17
18 #What prices are each door
19 Prices = {"Alu":3000, "Wood":5000}
20
21 #What is the time usage for framing and assembling
22 Frame_time = {"Alu":1, "Wood":2}
23 Assemble_time = {"Alu":3, "Wood":2}
24
25 #How much time do we have for each process
26 Max_Assemble = 10
27 Max_frame = {"Alu":4, "Wood":12}
28
```

```
30 #We establish a model variable -> model
31 model = pyo.ConcreteModel()
32
```

# Pyomo – Working with setup

- Declare sets
  - Contains list of objects
  - Ex: Door types, power plants, batteries etc..
  - Can shorten down code complexity drastically
- Declare Parameters
  - Contains the values given
  - Can have a dimension of the sets, or be dimensionless

```
33 """
34 Specify sets
35 """
36
37 model.Doors = pyo.Set(initialize = Doors)
38 #Note, in ConcreteModel we initialize a declaration like this with a variable already specified
39 #So we fill the set model.Doors with the data from Doors, in other words the list containing type of doors
40
41 """
42 Specify parameters
43 """
44
45 model.Prices = pyo.Param(model.Doors, initialize = Prices)
46 #Note the 'model.Doors', that is because we have parameters based on the list
47 #This is something to include when we speak of parameters with dimensions [this case, 2x1]
48
49 model.Frame_time = pyo.Param(model.Doors, initialize = Frame_time)
50 model.Assemble = pyo.Param(model.Doors, initialize = Assemble_time)
51
52 model.Max_Assemble = pyo.Param(initialize = Max_Assemble)
53 #Note here that since the dimension is 1x1, we don't include the set variable model.Doors
54
55 model.Max_frame = pyo.Param(model.Doors, initialize = Max_frame)
56
```

# Pyomo – Working with setup

- Declare variable
  - Specify the variables that we want to optimize
  - Can be dimensionless or be based on sets
  - Can specify boundaries in the declaration (optional, but can replace constraints later)
    - Can set it to be binary  $\{0,1\}$
    - Can set it to be integer  $\{0,1,2,3,\dots,N\}$
    - Etc

```
57 ***
58 Specify variables
59 ***
60
61 model.prod = pyo.Var(model.Doors, within = pyo.NonNegativeReals)
62 #For variables, we also include the set if it has multiple dimensions
63 #We could have created a separate variable for each door type, but this is easier and simpler
64 #Note the 'within', this allows us to specify the boundaries of the variable
65 #This is not necessary, one can specify it as a constraint instead
66
```

# Pyomo – Declare the Objective

- Objective declared through a function
  - Note: Must have the model as input
  - Return what the objective function is (note the sum used)
- For specifying the objective in 'model', one specifies:
  - «rule»: what depicts the objective (write function name)
  - «sense»: What type of optimization is this (min/max)

Objective without  
comments

```
74 def Obj(model):
75 return(sum(model.prod[k]*model.Prices[k] for k in model.Doors))
76 model.OBJ = pyo.Objective(rule = Obj, sense = pyo.maximize)
```

```
68 """
69 Specify Objective
70 """
71
72 #We need to create a function for this to work, that must include the model variable
73
74 def Obj(model):
75 #It must return the objective function
76 return(sum(model.prod[k]*model.Prices[k] for k in model.Doors))
77 #here, we specify the objective into the model variable
78 model.OBJ = pyo.Objective(rule = Obj, sense = pyo.maximize)
79 #sense specifies what we want to do (max/min). If not declared, it assumes max
80 #rule specifies what function that operates as the objective function
```

# Pyomo – How to create a constraint

- Create a function specifying the constraint
  - Use of  $\leq$ ,  $\geq$  or  $=$  to specify boundary
- To create constraint in 'model', link the constraint with 'rule'
- Note: Dimensionless or not creates multiple variations
- Can include dimension to create multiple constraints in one declaration

Dimensionless constraint, however the 'sum' is used to include both variables

```
100 def MTime(model):
101 return(sum(model.prod[n]*model.Assemble[n] for n in model.Doors) <= model.Max_Assemble)
102 model.MTime_const = pyo.Constraint(rule = MTime)
103
```

One function that declares two constraints due to:

- the input 'n'
- 'model.Doors' declared

in line 107

```
104 #Constraint for frame time, combine two constraints
105 def FTime(model,n):
106 return(model.prod[n]*model.Frame_time[n] <= model.Max_frame[n])
107 model.FTime_const = pyo.Constraint(model.Doors, rule = FTime)
108 #Note the 'n' specified in the function input. This n works as a list
109 #We use this 'n' in the return function, and when specifying the constraint
110 #We include the set model.Doors, which then tells the model that this constraint
111 #Operates for each value in the list model.Doors
112 #So, the constraint will here be 'run' twice, one for Alu, and one for Wood
113 #Thus, we save time to create an additional constraint
114
```

# Pyomo – Different solvers

- Many solvers exist, and can handle different problems
  - GLPK – Open-source solver, not the best and not very advanced
  - Gurobi – Need license (student license available for free). Very advanced and highly recommended (<http://www.gurobi.com/index>)
  - CPLEX – Need license (student license available for free?). Can handle advanced problems (<https://www.ibm.com/analytics/cplex-optimizer>)
- Packages needed for the solvers:
  - GLPK: `conda install -c conda-forge glpk`
  - Gurobi: `conda install -c gurobi gurobi`
  - Cplex: `conda install -c ibmdecisionoptimization cplex`
  - Note: Packages can be downloaded without license, but trying to use them without is troubling



# Pyomo – How to set a solver and run

- Set a solver
  - Here, GLPK is used
- Enable dual values
  - Good for further analysis, but optional
- Run the problem
  - Solve the problem to get optimal solutions

```
115 ***
116 Specify solver
117 ***
118
119 #We here decide what solver to use, note that you probably need packages for the solvers as well
120
121 #This simple model we use glpk
122 opt = SolverFactory('glpk')
123
124 ***
125 Enable dual values to be stored
126 ***
127
128 #One nice feature is to include extracting dual values. Done with this one line
129 model.dual = pyo.Suffix(direction=pyo.Suffix.IMPORT)
130
131 ***
132 Solve the problem
133 ***
134
135
136 #Solve the problem
137 results = opt.solve(model, load_solutions = True)
138
```

# Pyomo – How to work with results

*Write this to print out results*

- Data will be stored in the 'model' variable
- Can get them using .value
- Multiple methods to store the data, which can then be extracted to whatever usage wanted

```
139 #display data
140 model.display()
141 #display dual data
142 model.dual.display()
143
144 ***
145 Handling data afterwards
146 ***
147
148 #If we call for the variables, we will get the data right away:
149 for n in model.Doors:
150 print("number of " + n + " doors produced: " + str(model.prod[n].value))
151 print("Optimal solution: " + str(model.OBJ()))
152
153
```

Variable data

Constraint data

Dual data

```
In [20]: runfile(' //home.ansatt.ntnu.no/kasperet/Documents/Sci-Ass Courses/2019/TET4185/P
home.ansatt.ntnu.no/kasperet/Documents/Sci-Ass Courses/2019/TET4185/Python-Pyomo/Python 1
Model unknown

Variables:
prod : Size=2, Index=Doors
Key : Lower : Value : Upper : Fixed : Stale : Domain
Alu : 0 : 2.0 : None : False : NonNegativeReals
Wood : 0 : 6.0 : None : False : False : NonNegativeReals

Objectives:
OBJ : Size=1, Index=None, Active=True
Key : Active : Value
None : True : 36000.0

Constraints:
MTime_const : Size=1
Key : Lower : Body : Upper
None : None : 18.0 : 18.0
FTime_const : Size=2
Key : Lower : Body : Upper
Alu : None : 2.0 : 4.0
Wood : None : 12.0 : 12.0
dual : Direction=Suffix.IMPORT, Datatype=Suffix.FLOAT
Key : Value
FTime_const[Alu] : 0.0
FTime_const[Wood] : 1500.0
MTime_const : 1000.0
number of Alu doors produced: 2.0
number of Wood doors produced: 6.0
Optimal solution: 36000.0
```

# Pyomo – positives with this setup

- By not declaring any direct values in the model, one can change the input values, and the whole problem is automatically changed
- Example: Add a steel door into input data
  - The model is structured so that no additional adjustment in the model is necessary! Time saved!

```
15 #What type of doors do we have
16 Doors = ["Alu", "Wood", "Steel"] #Note, this is a list with the type of door
17
18 #What prices are each door
19 Prices = {"Alu":3000, "Wood":5000, "Steel":5200} #Note, this is a dictionary specifying price for given door
20
21 #What is the time usage for framing and assembling
22 Frame_time = {"Alu":1, "Wood":2, "Steel":1} #Note, this is also a dictionary
23 Assemble_time = {"Alu":3, "Wood":2, "Steel":1.5} #Note, also a dictionary
24
25 #How much time do we have for each process
26 Max_Assemble = 18 #Note, not a dictionary, but a constant
27 Max_frame = {"Alu":4, "Wood":12, "Steel":4.5} #Dictionary
```

```
Model unknown

Variables:
 prod : Size=3, Index=Doors
 Key : Lower : Value : Upper : Fixed : Stale : Domain
 Alu : 0 : 0.0 : None : False : False : NonNegativeReals
 Steel: 0 : 4.5 : None : False : False : NonNegativeReals
 Wood : 0 : 5.625 : None : False : False : NonNegativeReals

Objectives:
 OBJ : Size=1, Index=None, Active=True
 Key : Active : Value
 None : True : 51525.0

Constraints:
 MTime_const : Size=1
 Key : Lower : Body : Upper
 None : None : 18.0 : 18.0
 FTime_const : Size=3
 Key : Lower : Body : Upper
 Alu : None : 0.0 : 4.0
 Steel: None : 4.5 : 4.5
 Wood : None : 11.25 : 12.0
dual : Direction=Suffix.IMPORT, Datatype=Suffix.FLOAT
 Key : Value
 FTime_const[Alu] : 0.0
 FTime_const[Steel] : 1450.0
 FTime_const[Wood] : 0.0
 _MTime_const : 2500.0
number of Alu doors produced: 0.0
number of Wood doors produced: 5.625
number of Steel doors produced: 4.5
Optimal solution: 51525.0
```

# Pyomo – Endless possibilities

- Almost any problem can be solved with this setup
- Practice makes perfect
  - Start simple, find shortcuts
- There will be problems in this course that can be solved with this method, some on the exercises
- An exercise created for the 5th year students will be included as optional practice material



**Questions, comments? 😊**



# Pyomo – Different solvers

- Many solvers exist, and can handle different problems
  - GLPK – Open-source solver, not the best and not very advanced
  - Gurobi – Need license (student license available for free). Very advanced and highly recommended (<http://www.gurobi.com/index>)
  - CPLEX – Need license (student license available for free?). Can handle advanced problems (<https://www.ibm.com/analytics/cplex-optimizer>)
- Packages needed for the solvers:
  - GLPK: `conda install -c conda-forge glpk`
  - Gurobi: `conda install -c gurobi gurobi`
  - Cplex: `conda install -c ibmdecisionoptimization cplex`
  - Note: Packages can be downloaded without license, but trying to use them without is troubling